

Bachelor's Thesis

Implementing Floating-Point Support in a Nanopass-Python Compiler

Marius Spill

Examiner: Prof. Dr. Peter Thiemann

Advisers: TO DO

Albert-Ludwigs-University Freiburg

Faculty of Engineering

Department of Computer Science

Chair for Programming Languages

September 15, 2026

Writing period

06.15.2026 – 09.15.2026

Examiner

Prof. Dr. Peter Thiemann

Advisers

TODO

Declaration

I hereby declare that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I hereby also declare that my Thesis has not been prepared for another examination or assignment, either wholly or excerpts thereof.

Place, Date

Signature

Abstract

foo bar

Contents

1	Introduction	1
1.1	Subchapter 1	1
1.2	Subchapter 2	1
2	Background	3
3	Design Architecture	5
4	Implementation	7
4.1	Interpreter Extension and Phase-Driven Design	7
4.1.1	Syntactic Extensions (The Parser)	8
4.1.2	Static Semantics and the 64-Bit Precision Decision . .	8
4.1.3	Type-System Generalization	9
4.1.4	Interpreter Semantics	9
4.2	Extending the compiler to take literals through the whole pipeline	10
5	Evaluation	11
6	Conclusion	13
	Bibliography	15

1 Introduction

1.1 Subchapter 1

The Hook: Why do we care about small/incremental compiler frameworks (like Nanopass)?

1.2 Subchapter 2

The Problem: What happens when a framework or downstream intermediate representation (IR) lacks native floating-point types? (e.g., restriction to purely integer arithmetic limits the computational expressiveness of the language). The Contribution: A brief high-level summary of what you added to the pipeline.

2 Background

The Nanopass Framework: Explain how the host compiler operates (small, discrete passes transforming highly specific AST micro-languages). Floating-Point Representation: Briefly introduce the target representation semantics (e.g., IEEE 754 standards) and how the target machine or runtime expects floats to be structured compared to integers.

3 Design Architecture

AST Extensions: What new node types did you introduce to represent float literals, float primitives, and operators? Pass Pipeline Changes: Walk the reader through the compilation pipeline. Which existing passes had to change to propagate float types, and what new passes did you have to insert to deal with float semantics?

4 Implementation

The integration of floating-point operations requires a full-vertical extension across the compiler pipeline. This chapter details the engineering semantics, type systems, and lowering mechanisms required to support 64-bit double-precision floats within a base 32-bit architecture.

4.1 Interpreter Extension and Phase-Driven Design

Before implementing the concrete compiler backend, the language’s dynamic semantics were established by extending the reference interpreter. Because the interpreter defines the ground-truth behavioral specification of the language, this execution layer must be finalized first to ensure semantic parity with the compiler’s eventual target output.

To establish clear verification metrics, a differential test suite containing half a dozen Python programs was constructed. These test cases duplicated existing integer-based control flow, arithmetic expressions, and native built-in functions (such as `print()`), substituting integer literals with floating-point values. The immediate implementation goal was to allow the interpreter to execute this test suite natively without throwing exceptions, matching Python’s runtime behavior exactly.

Extending the interpreter required systematic modifications across three core subsystems of the pipeline: the abstract syntax tree (AST) parser, the static type-checker, and the evaluation semantics engine.

4.1.1 Syntactic Extensions (The Parser)

Extending the frontend parser was highly deterministic because the host language's `ast` module natively recognizes floating-point tokens. The internal AST grammar definition was upgraded to allow literal expression nodes (`EConst`) to hold float primitives alongside integers. Additionally, the type-mapping routines were extended to recognize and propagate explicit variable type annotations, allowing patterns such as `a : float = 3.14` to pass successfully through the front-end token stream.

4.1.2 Static Semantics and the 64-Bit Precision Decision

The primary design challenge emerged during the static analysis phase within the type-checking subsystem. Because the compiler targets a 32-bit RISC-V architecture (RV32), implementing 32-bit single-precision floats (the RISC-V "F" extension) would have offered significant implementation expediency; 32-bit floats map cleanly to standard 32-bit stack slots and integer register pairs.

However, architectural integrity took precedence over implementation simplicity. Python natively treats all floating-point numbers as 64-bit doubles. Forcing a static 32-bit truncation at the type-checker level would introduce severe semantic drift from standard Python behavior, causing silent precision loss on common practical decimals like `0.1` or `3.14`.

A hybrid architecture—selectively flagging exact base-2 powers (e.g., $0.5 = 2^{-1}$) as 32-bit floats and fallback decimals as 64-bit doubles—was also explored and ultimately rejected. Python does not natively support or expose mixed-precision literals; forcing such an abstraction would create an un-Pythonic dialect. Furthermore, it would introduce significant complexity into the backend code generator, requiring exhaustive permutations of type-coercion routines and hardware conversion instructions for cross-type arithmetic operations. To preserve strict language fidelity, a unified 64-bit

double-precision model was selected, targeting the RISC-V Double-Precision ("D") extension. Consequently, the type validator was designed to enforce a single constraint: verifying that literals do not exceed the 64-bit IEEE 754 infinity threshold.

4.1.3 Type-System Generalization

Beyond validating literals, the type-checker's expression evaluation routines needed a clean refactoring. Originally, the compiler assumed almost everything was an integer, evaluating binary operators and functions like `print()` by checking them directly against a hardcoded `TInt()` type. To support floating-point numbers without copying and pasting the same validation logic everywhere, this rigid approach was replaced with a more flexible system. A new helper function, `check_types_supported`, was introduced to validate expressions against a list of acceptable types instead of just one. For this first iteration, operands must still match exactly—meaning mixed-type operations like `float + int` are blocked for now. However, this refactor ensures that expressions evaluate to and return their correct type primitive (like `TFloat()`) rather than defaulting to an integer, laying the clean groundwork for future cross-type arithmetic.

4.1.4 Interpreter Semantics

Extending the runtime behavior in the interpreter was the most straightforward step because the interpreter itself is written in Python. Since the host language already supports 64-bit floating-point math natively.

In `semantics.py`, the evaluation rules for binary expressions were simply updated to accept float values alongside integers. When the interpreter encounters an operation like addition, it handles the AST nodes by passing the raw values directly to Python's native operators. By leveraging the host language's execution environment this way, the interpreter was immediately

able to run the floating-point test suite with perfect accuracy.

4.2 Extending the compiler to take literals through the whole pipeline

- `pass_1_2_shrink`: Had to add a 64-bit flag if constant is float so down the line its exact ieee structure isnt lost.
- `pass_1_2_shrink`: Else the pass didnt have to be adjusted for floats (maybe only the ast adding float)

To be implemented !

5 Evaluation

Correctness Validation: Instead of standard benchmarks or accuracy percentages, compiler evaluations validate determinism and compliance. Explain your test suite framework. Test Cases: Discuss how you validated corner cases (e.g., parsing precision limits, type coercion, complex floating-point expressions). Performance (Optional): If you optimize passes or look at compilation times, present it here. Otherwise, focus heavily on functional correctness and compilation validation.

6 Conclusion

Short summary of your execution and a couple of bullet points on future work (e.g., supporting double-precision, floating-point optimization passes, or array allocations of floating-point numbers).

Bibliography

